Mills Museum AI Keyword Generation System

Abstract

This project presents an end-to-end system for automating keyword annotation of museum artworks using the Getty Art and Architecture Thesaurus. The system integrates data engineering, semantic embeddings, approximate nearest neighbor search, clustering, and a human-in-the-loop user interface. Emphasis is placed on data quality, reproducibility, and extensibility,, so future contributors can onboard quickly and improve components independently.

Link to video explanation can be found: Code walkthroughs

1. System Overview

The project is organized as a modular pipeline that transforms vocabulary data and museum images into validated keyword outputs. Art and Architecture Thesaurus data is first cleaned and deduplicated to form a standardized vocabulary. Museum images are then processed by a keyword generation model, matched against the controlled vocabulary, and reviewed in the user interface before final selection.

The system is built around three design principles: separation of concerns between data, model, and interface layers; deterministic preprocessing for reproducibility; and human-in-the-loop validation for quality control.

Pipeline Diagram

```
AAT Data -> Cleaning -> Deduplication -> Clean Vocabulary
                                          |
                                          v
Images -> Embedding Model -> Raw Keywords -> AAT Matching ->
User Interface
```

2. Setup and Prerequisites

To run the backend locally, contributors need Python 3.10 or later and libraries such as datasets, pandas, pyarrow, sentence-transformers, Facebook AI Similarity Search, numpy, and torch. To run the frontend locally, contributors need Node.js and npm for the React-based user interface.

A typical local workflow is: load the dataset, run the semantic deduplication pipeline, prepare museum data, and then launch the frontend. Running the pipeline in this order ensures that cached data, cleaned vocabularies, and expected intermediate outputs exist before a contributor starts testing the user interface.

```
python load_dataset.py
 python semantic_dedup_pipeline.py
 python data_to_hf.py
 # then launch the frontend development server
```

3. Backend Architecture

3.1 Data Layer

The data layer is responsible for loading, caching, and transforming the Art and Architecture Thesaurus and museum metadata into formats that support analysis and modeling.

load_dataset.py

This module downloads the Art and Architecture Thesaurus dataset from Hugging Face and caches it locally as a parquet file. Parquet was selected because it provides faster input and output operations and more efficient storage than plain-text formats such as comma-separated values files.

File: load_dataset.py

```
from datasets import load_dataset

dataset = load_dataset("aat_museum_subset")
 df = dataset["train"].to_pandas()
 df.to_parquet("data_cache/aat.parquet")
```

Alternative considered: comma-separated values files. This was not selected because plain-text parsing is slower, file sizes are larger, and repeated development iterations become less efficient.

data_to_hf.py

This module converts museum metadata into a machine learning-ready dataset and links museum images to records using accession numbers. A flexible file-matching strategy was selected because museum image naming is not always perfectly uniform.

File: data_to_hf.py

```
# Match images flexibly using accession number prefixes
 matches = glob.glob(f"data/images/{accession_number}*.png")
```

Alternative considered: enforcing a strict naming convention. This was not selected because real museum collections often contain inconsistent file naming, and a rigid approach would create fragile preprocessing.

3.2 Semantic Deduplication Pipeline

The semantic deduplication pipeline is the core backend contribution of the project. Its purpose is to reduce redundant or near-duplicate controlled vocabulary labels so downstream matching becomes more consistent and less noisy.

Step 1: Language Normalization

Language normalization ensures that missing or null language identifiers are handled consistently and that semantically similar labels are not incorrectly merged across languages.

File: semantic_dedup_core.py

```
# Prevent cross-language merging by normalizing missing language
values
 LANG_NULL_TOKEN = "__NULL_LANGUAGE__"
 df["language_id"] = df["language_id"].fillna(LANG_NULL_TOKEN)
```

Step 2: Embedding Generation

Each label is converted into a dense vector embedding using a SentenceTransformers model. The model all-MiniLM-L6-v2 was selected because it offers a good balance between speed and semantic quality for a project of this scale.

File: semantic_dedup_core.py

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

# normalize_embeddings=True allows cosine similarity to be
computed with dot product
 embeddings = model.encode(labels, normalize_embeddings=True)
```

Alternative considered: larger transformer models such as MPNet. This was not selected because the latency and computing costs were higher than necessary for the size of this dataset.

Step 3: Similarity Search

The project uses Facebook AI Similarity Search to find nearest neighbors efficiently in embedding space. Because embeddings are normalized, inner product search corresponds to cosine similarity.

File: semantic_dedup_core.py

```
import faiss

index = faiss.IndexFlatIP(embeddings.shape[1])
 index.add(embeddings)

# Find nearest neighbors for each label
 sims, neighbors = index.search(embeddings, k=10)
```

Alternative considered: brute-force pairwise search. This was not selected because it scales poorly and becomes computationally expensive as the number of labels grows.

Step 4: Clustering

Similarity edges above a threshold are merged using a Union-Find data structure. This supports flexible connected-component clustering without requiring a fixed number of clusters.

File: semantic_dedup_core.py

```
# Merge labels into the same cluster if similarity exceeds the
threshold
 if sims[i, j] >= threshold:
     uf.union(i, j)
```

Alternative considered: K-Means clustering. This was not selected because the number of clusters is not known in advance and cluster sizes are naturally uneven. Alternative considered: hierarchical clustering. This was not selected because it is more expensive in time and memory for this use case.

Step 5: Canonical Label Selection

After clusters are formed, the system selects a canonical label deterministically. This matters because clustering alone is not sufficient; each group still needs a representative label for downstream use.

File: semantic_dedup_core.py

```
# Select canonical label from a cluster
 canonical = sorted(cluster, key=lambda x: (
     not has_note[x],   # prefer labels with descriptions
     len(x),            # prefer shorter labels
     x                  # alphabetical fallback
 ))[0]
```

This strategy prioritizes labels with descriptions, then shorter labels, then alphabetical order. The goal is to balance informativeness, readability, and determinism.

Step 6: Metrics and Outputs

The pipeline evaluates quality using semantic collision rate and token entropy. It also writes multiple files so contributors can inspect both the final vocabulary and the transformation process.

File: semantic_dedup_core.py

```
deduped.to_parquet(args.output_parquet, index=False)
 audit_df.to_csv(args.audit_csv, index=False)
 mapping_df.to_csv(args.mapping_csv, index=False)

print(f"Saved deduplicated parquet: {args.output_parquet}")
 print(f"Saved audit CSV: {args.audit_csv}")
 print(f"Saved mapping CSV: {args.mapping_csv}")
```

Deduplication Workflow Diagram

```
Labels -> Embeddings -> Similarity Search -> Filter by Threshold
-> Clustering -> Canonical Selection -> Clean Vocabulary
```

4. Backend and Application Programming Interface

The backend supports both the **data-processing pipeline** and the **keyword-generation interface**. At a high level, it performs four responsibilities: loading and preparing controlled vocabulary data, deduplicating semantically similar labels, serving keyword predictions for

uploaded images, and returning structured results that the frontend can render and export. The semantic deduplication pipeline remains the core backend data-cleaning stage. It embeds labels, clusters semantically similar terms within each language, chooses canonical labels, and writes three outputs: a cleaned parquet file, an audit comma-separated values file, and a term-level mapping comma-separated values file. The updated implementation also contains more explicit internal documentation, including comments explaining dependency checks, vector normalization, language isolation, cluster previews, and why multiple output files are written for different audiences.

The command-line entrypoint for the deduplication stage is intentionally lightweight. The wrapper script exists so contributors can run the pipeline directly while keeping the main implementation importable for testing and reuse. That design improves maintainability because the executable script stays simple while the core logic remains modular.

For interactive keyword generation, the backend exposes a POST /predict application programming interface endpoint. The React/Vite browser app sends uploaded images to this endpoint over Hypertext Transfer Protocol, and the response is converted into user-interface-friendly keyword objects by shared adapters in the frontend utility layer. The backend output supports downstream review operations such as inclusion or exclusion of keywords, export to text or comma-separated values, and confidence display.

File: src/frontend/web/src/App.jsx

```
const formData = new FormData()
formData.append('file', file)
formData.append('term_count', String(termCount))

const res = await fetch(`${API_URL}/predict`, {
     method: 'POST',
     headers: { 'ngrok-skip-browser-warning': 'true' },
     body: formData,
})

const data = await res.json()
```

A typical response is a JavaScript Object Notation object containing a list of keyword predictions and confidence values. Error handling should return appropriate status codes for invalid files, missing inputs, or model failures.

```
{

  "keywords": [
```

```
    { "text": "historic landscapes", "confidence": 51.8 },
    { "text": "atmospheric perspective", "confidence": 51.4 },
    { "text": "landscapists", "confidence": 51.0 }
  ]
}
```


Backend Design Note

A major architectural idea in the Python/Gradio workflow is that the user interface layer does **not** define the machine learning models itself. Instead, runtime model components such as the embedding model, reranking model, and vector collection are expected to be initialized elsewhere and injected into the interface. This separation keeps the user interface orchestration independent from the model runtime and makes the application easier to test and extend.

5. Frontend Architecture

The frontend is now more advanced than the earlier upload-process-review description and should be documented as a **configurable keyword review environment** rather than only a simple result screen. The repository currently contains two user-facing keyword-review implementations: a **Python/Gradio workflow** in src/frontend/gradio.py and a **React/Vite browser application** in src/frontend/web/. There is also a third interface, the Gradio-based data exhibit in src/frontend/data_viz/data_story_exhibit.py, but that tool is for exploring the Art and Architecture Thesaurus dataset rather than reviewing image keywords.

The React/Vite application still follows the same broad product flow — **upload, processing, and review** — but the review stage now includes several added controls and visualization options. The updated user interface shows: a slider for the **number of keywords to generate**, a **query bias** control that balances image information against description information, a **diversity (Maximal Marginal Relevance)** control that adjusts the tradeoff between diversity and pure relevance, a visible **Artificial Intelligence description** panel during processing, and review tools that support **grouped or hierarchy-based presentation**, **heatmap view**, filtering, copying, and multiple export formats. These newer behaviors are visible in the updated screenshots.

Query Bias

This setting controls how much weight is given to the uploaded image versus the generated or provided description when producing keyword suggestions. A value near zero favors image evidence, while a value near one favor descriptive text. The default centered value reflects a balanced query. The updated settings screen explicitly shows the slider running from “Image” to “Description.”

Diversity (Maximal Marginal Relevance)

This setting controls whether the output list should prioritize a broader range of keywords or concentrate on highly similar, highly relevant terms. A lower value encourages variety, while a higher value pushes the results toward pure relevance. The user interface currently shows this as a slider from “Diverse” to “Relevant,” with a default close to relevance-heavy output.

Artificial Intelligence Description

During the processing stage, the interface now surfaces with an Artificial Intelligence-generated textual description of the artwork while progress continues. This is important because it gives the user a readable explanation of what the system is perceiving before the final keyword list is shown. The processing screen explicitly displays a description card beside the image preview.

Grouped/Hierarchy-Based Review

The review screen now supports a grouped layout where suggested keywords are organized under higher-level buckets such as Built Environment, Color, Components, Living Organisms, and Materials. This makes the output easier to inspect because related terms are visually clustered rather than shown only as one flat list. The screenshots clearly show grouped keyword sections in the updated interface.

Heatmap view

The review interface also exposes a heatmap mode, visible as a top-level control next to Group, Copy, text export, and comma-separated values export. In the context of the user interface, this feature provides an alternate visualization mode for inspecting keyword patterns rather than only reading them in a flat or grouped list.

Frontend Workflow Diagram

```
Upload → Configure generation settings → Process image

       → View Artificial Intelligence description

       → Review keywords in flat, grouped, or heatmap-oriented
     views
```

```
→ Filter, include or exclude, copy, or export
```

To run the frontend locally, contributors should navigate to the frontend directory, install dependencies with npm install, and start the development server.

6. Visualization Layer

The visualization layer provides insight into the system's behavior using techniques such as Principal Component Analysis, heatmaps, word clouds, and network graphs. These visualizations help identify clustering quality, keyword relationships, and potential dataset biases.

Implementation Examples

File: 06_advanced_analysis.py

```
from sklearn.decomposition import PCA
 import matplotlib.pyplot as plt

pca = PCA(n_components=2)
 points_2d = pca.fit_transform(scope_matrix.toarray())

plt.scatter(points_2d[:, 0], points_2d[:, 1], s=8, alpha=0.5)
 plt.xlabel("Principal Component 1")
 plt.ylabel("Principal Component 2")
 plt.title("PCA Projection of Scope Note Embeddings")
 plt.show()
```

File: 04_visualizations.py

```
import seaborn as sns
 import matplotlib.pyplot as plt

sns.heatmap(heatmap_df, annot=True, cmap="YlOrRd")
 plt.title("Century References by Facet")
 plt.xlabel("Century")
 plt.ylabel("Facet")
 plt.show()
```

Visualization Workflow Diagram

```
Dataset -> Feature Extraction -> Dimensionality Reduction ->
Visualization
                                                |-> Principal
Component Analysis
                                                |-> Heatmap
                                                |-> Word Cloud
                                                |-> Network Graph
```

7. Onboarding and Expected Outputs

New contributors should follow a structured workflow when onboarding. At each stage they should verify that expected files or interface outputs are produced. This reduces debugging time and helps isolate configuration problems early.

Verification Examples

File: load_dataset.py

```
df = load_aat_dataset()
 print(f"Shape: {df.shape}")
 print(f"Columns: {list(df.columns)}")
```

Expected output: a dataset shape and a list of expected schema columns.

File: semantic_dedup_core.py

```
print(f"Saved deduplicated parquet: {args.output_parquet}")
 print(f"Saved audit CSV: {args.audit_csv}")
 print(f"Saved mapping CSV: {args.mapping_csv}")
```

Expected output: confirmation that deduplicated parquet, audit comma-separated values file, and mapping comma-separated values file were written.

File: frontend keywordAdapters.js

```
console.log(data.keywords)
```

```
{
   "keywords": [
     { "text": "modern", "confidence": 97 },
     { "text": "sculpture", "confidence": 92 }
   ]
 }
```

Onboarding Workflow Diagram

```
Load Dataset -> Verify Parquet
        |
        v
 Run Deduplication -> Verify Deduplicated Outputs
        |
        v
 Prepare Museum Data -> Verify Clean Dataset
        |
        v
 Run Frontend -> Verify Keywords Appear in User Interface
```

8. Documentation Consistency and Clarity

Documentation should remain consistent in both terminology and code presentation. For example, the term user interface should be introduced once and then reused consistently. Code snippets should always indicate their file location, include short explanatory comments where needed, and show expected outputs whenever possible.

File: semantic_dedup_core.py

```
# Normalize missing language values so clustering remains
language-aware
 LANG_NULL_TOKEN = "__NULL_LANGUAGE__"
 df["language_id"] = df["language_id"].fillna(LANG_NULL_TOKEN)
```

Best practices for this project include labeling all code snippets with their source file, keeping terminology consistent, and including screenshots or example outputs for complex interface or visualization sections.

9. Design Decisions and Alternatives Considered

Several implementation decisions were made intentionally to balance speed, clarity, scalability, and maintainability.

MiniLM was chosen for embeddings because it is fast and efficient, while larger models were not selected due to unnecessary latency. Facebook AI Similarity Search was chosen for scalable similarity search, while brute-force search was not selected due to poor computational scaling. Union-Find clustering was chosen because it supports flexible cluster sizes and does not require predefining the number of clusters, while K-Means and hierarchical clustering were not selected because they were a poor fit for the data structure and performance requirements. React was chosen for the frontend because it supports a more interactive user experience, even though it introduces more setup complexity than simpler templating approaches.

10. Limitations

The system still has important limitations. Some model-generated keywords may be inaccurate. Cross-language semantic merging is not currently supported in the deduplication pipeline. In addition, embeddings are not yet fine-tuned specifically on the Art and Architecture Thesaurus domain.

11. Future Work

Future improvements could include fine-tuning embeddings on the Art and Architecture Thesaurus corpus, integrating stronger reranking models, adding multimodal image-and-text matching, and creating a feedback loop so user corrections improve future predictions.

12. How to Onboard and Contribute

A new contributor should begin by reading the backend architecture section, then studying semantic_dedup_core.py because it contains the core algorithmic logic. From there, they should run the project in a staged way: load the dataset, run deduplication, prepare museum data, and then test the frontend.

The most important starting points are semantic_dedup_core.py for core logic, data_to_hf.py for data preparation, and the frontend directory for interface behavior. Contributors who want to

improve the model should start in the deduplication and matching pipeline; contributors who want to improve usability should begin in the React components.

Common pitfalls include incorrect image-to-accession matching, low-confidence keyword predictions, and incorrect clustering caused by inconsistent language normalization.

13. Conclusion

This system demonstrates how structured vocabulary and artificial intelligence can be combined to automate complex museum tagging workflows. Its modular architecture allows components to be improved independently, while the inclusion of human validation preserves interpretability and quality. As a result, the project provides both a practical tool for museum workflows and a strong foundation for future contributors.